

C++ 输入输出与文件处理完整自学手册

目录

1. C++ 输入输出概述
 2. 流和缓冲区基础
 3. 使用cout进行输出
 4. 使用cin进行输入
 5. 文件输入和输出
 6. 内核格式化
 7. 实战练习
 8. 常见问题解答
-

1. C++ 输入输出概述

1.1 为什么学习C++ 输入输出？

C++与许多编程语言不同，它没有将输入输出功能直接构建在语言中。相反，C++使用了一套强大的类库来实现输入输出功能。这套类库提供了：

- 类型安全的输入输出
- 可扩展的格式化选项
- 统一的接口处理不同数据源
- 缓冲机制提高性能

1.2 C++ 输入输出的核心概念

关键术语：

- **流 (Stream)**：数据的序列，可以是输入流或输出流
- **缓冲区 (Buffer)**：临时存储数据的内存区域
- **格式化**：将数据转换为特定格式进行显示或存储

主要类层次结构：

```
ios_base (基类)
  ↓
ios
  ↓
istream ostream
  ↓   ↓
iostream
  ↓
fstream
```

2. 流和缓冲区基础

2.1 什么是流？

流是程序与外部世界交换数据的抽象概念。想象一下水管中的水流：

- 数据像水一样从源头流到目标
- 程序从输入流中"抽取"数据
- 程序向输出流中"插入"数据

2.2 缓冲区的作用

缓冲区就像水库，它的作用是：

1. **匹配速度差异**：程序处理数据的速度与设备读写速度不同
2. **提高效率**：批量传输比逐字节传输更高效
3. **临时存储**：在数据到达最终目标之前暂时保存

缓冲区工作示例：

```
cpp

#include <iostream>
using namespace std;

int main() {
    // 输出会被缓冲，直到遇到换行符或程序结束
    cout << "这行文字会被缓冲";
    cout << "这行也会被缓冲";
    cout << "这行有换行符\n"; // 这时才会显示所有内容

    return 0;
}
```

2.3 标准流对象

C++ 自动创建了8个流对象：

对象名	类型	作用	是否缓冲
<code>cin</code>	<code>istream</code>	标准输入（键盘）	是
<code>cout</code>	<code>ostream</code>	标准输出（屏幕）	是
<code>cerr</code>	<code>ostream</code>	标准错误（屏幕）	否
<code>clog</code>	<code>ostream</code>	标准日志（屏幕）	是
<code>wcin</code>	<code>wistream</code>	宽字符输入	是
<code>wcout</code>	<code>wostream</code>	宽字符输出	是
<code>wcerr</code>	<code>wostream</code>	宽字符错误	否
<code>wclog</code>	<code>wostream</code>	宽字符日志	是

3. 使用cout进行输出

3.1 基本输出操作

3.1.1 插入运算符 <<

`<<` 运算符是输出的核心，它被重载以支持所有基本数据类型：

```
cpp
#include <iostream>
using namespace std;

int main() {
    int age = 25;
    double height = 175.5;
    char grade = 'A';
    string name = "张三";

    cout << "姓名: " << name << endl;
    cout << "年龄: " << age << endl;
    cout << "身高: " << height << " cm" << endl;
    cout << "等级: " << grade << endl;

    return 0;
}
```

3.1.2 连接输出（链式调用）

cpp

```
cout << "年龄: " << age << ", 身高: " << height << " cm" << endl;
```

3.2 格式化输出

3.2.1 设置数制

cpp

```
#include <iostream>
using namespace std;

int main() {
    int num = 42;

    cout << "十进制: " << dec << num << endl; // 42
    cout << "八进制: " << oct << num << endl; // 52
    cout << "十六进制: " << hex << num << endl; // 2a

    // 显示进制前缀
    cout << showbase;
    cout << "带前缀的八进制: " << oct << num << endl; // 052
    cout << "带前缀的十六进制: " << hex << num << endl; // 0x2a

    return 0;
}
```

3.2.2 设置字段宽度

cpp

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    // 使用 setw() 设置字段宽度
    cout << "|" << setw(10) << "Name" << "|" << setw(5) << "Age" << "|" << endl;
    cout << "|" << setw(10) << "Alice" << "|" << setw(5) << 25 << "|" << endl;
    cout << "|" << setw(10) << "Bob" << "|" << setw(5) << 30 << "|" << endl;

    return 0;
}
```

输出：

```
|  Name| Age|  
|  Alice| 25|  
|   Bob| 30|
```

3.2.3 设置对齐方式

```
cpp  
  
#include <iostream>  
#include <iomanip>  
using namespace std;  
  
int main() {  
    cout << left; // 左对齐  
    cout << "|" << setw(10) << "Name" << "|" << setw(5) << "Age" << "|" << endl;  
  
    cout << right; // 右对齐 (默认)  
    cout << "|" << setw(10) << "Alice" << "|" << setw(5) << 25 << "|" << endl;  
  
    return 0;  
}
```

3.2.4 浮点数格式化

```
cpp
```

```

#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    double pi = 3.14159265359;

    // 设置精度
    cout << "默认: " << pi << endl;
    cout << "精度2: " << setprecision(2) << pi << endl;
    cout << "精度5: " << setprecision(5) << pi << endl;

    // 固定小数点格式
    cout << fixed << setprecision(3) << pi << endl; // 3.142

    // 科学计数法
    cout << scientific << setprecision(2) << pi << endl; // 3.14e+00

    return 0;
}

```

3.2.5 填充字符

```

cpp

#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    cout << setfill('*') << setw(10) << 42 << endl; // *****42
    cout << setfill('0') << setw(6) << 123 << endl; // 000123

    return 0;
}

```

3.3 其他输出方法

3.3.1 put() 方法

```

cpp

```

```

#include <iostream>
using namespace std;

int main() {
    cout.put('A');      // 输出单个字符
    cout.put(65);       // 输出ASCII码65对应的字符'A'
    cout.put('B').put('C'); // 链式调用

    return 0;
}

```

3.3.2 write() 方法

```

cpp

#include <iostream>
#include <cstring>
using namespace std;

int main() {
    char str[] = "Hello World";
    cout.write(str, 5); // 只输出前5个字符: "Hello"
    cout << endl;

    return 0;
}

```

3.4 控制缓冲区

```

cpp

#include <iostream>
using namespace std;

int main() {
    cout << "这行会立即显示" << flush; // 立即刷新缓冲区
    cout << "这行也会立即显示" << endl; // 输出换行符并刷新缓冲区

    // 手动刷新
    cout << "手动刷新";
    cout.flush();

    return 0;
}

```

4. 使用cin进行输入

4.1 基本输入操作

4.1.1 抽取运算符 >>

```
cpp

#include <iostream>
using namespace std;

int main() {
    int age;
    double salary;
    string name;

    cout << "请输入姓名: ";
    cin >> name;    // 注意：只能读取到第一个空格

    cout << "请输入年龄: ";
    cin >> age;

    cout << "请输入薪水: ";
    cin >> salary;

    cout << "您好, " << name << "! 您今年 " << age
        << " 岁, 薪水是 " << salary << endl;

    return 0;
}
```

4.1.2 链式输入

```
cpp

cout << "请输入三个数字: ";
int a, b, c;
cin >> a >> b >> c;
```

4.2 处理输入中的空白字符

4.2.1 get() 方法读取单个字符

```
cpp
```



```

#include <iostream>
using namespace std;

int main() {
    char ch;

    cout << "请输入一个字符: ";
    cin.get(ch);    // 读取单个字符，包括空白字符
    cout << "您输入的字符是: " << ch << " " << endl;

    // 另一种形式
    cout << "请再输入一个字符: ";
    ch = cin.get(); // 返回字符的ASCII码
    cout << "字符: " << ch << ", ASCII: " << (int)ch << endl;

    return 0;
}

```

4.2.2 get() 方法读取字符串

```

cpp

#include <iostream>
using namespace std;

int main() {
    char name[50];

    cout << "请输入您的全名: ";
    cin.get(name, 50); // 读取最多49个字符（保留位置给\0）
    cout << "您的全名是: " << name << endl;

    return 0;
}

```

4.2.3 getline() 方法

```

cpp

```

```

#include <iostream>
#include <string>
using namespace std;

int main() {
    string line;

    cout << "请输入一行文本: ";
    getline(cin, line); // 读取整行, 包括空格
    cout << "您输入的是: " << line << endl;

    // 对于C风格字符串
    char buffer[100];
    cout << "再输入一行: ";
    cin.getline(buffer, 100);
    cout << "C风格字符串: " << buffer << endl;

    return 0;
}

```

4.3 输入验证和错误处理

4.3.1 检查输入是否成功

```

cpp

#include <iostream>
using namespace std;

int main() {
    int number;

    cout << "请输入一个整数: ";
    if (cin >> number) {
        cout << "输入成功, 数字是: " << number << endl;
    } else {
        cout << "输入失败! " << endl;
        cin.clear();           // 清除错误标志
        cin.ignore(1000, '\n'); // 清除输入缓冲区
    }

    return 0;
}

```

4.3.2 安全的输入函数

cpp

```
#include <iostream>
#include <limits>
using namespace std;

int getInteger() {
    int number;
    while (true) {
        cout << "请输入一个整数: ";
        if (cin >> number) {
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            return number;
        } else {
            cout << "输入无效，请重试! " << endl;
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
        }
    }
}

int main() {
    int num = getInteger();
    cout << "您输入的数字是: " << num << endl;
    return 0;
}
```

4.4 流状态管理

4.4.1 流状态标志

cpp

```

#include <iostream>
using namespace std;

void showStreamState() {
    cout << "流状态: ";
    if (cin.good()) cout << "good ";
    if (cin.eof()) cout << "eof ";
    if (cin.fail()) cout << "fail ";
    if (cin.bad()) cout << "bad ";
    cout << endl;
}

int main() {
    int num;

    cout << "请输入一个整数: ";
    cin >> num;

    showStreamState();

    if (cin.fail()) {
        cout << "输入失败, 重置流状态" << endl;
        cin.clear();
        cin.ignore(1000, '\n');
    }

    return 0;
}

```

4.5 高级输入技术

4.5.1 peek() 和 putback()

cpp

```
#include <iostream>
using namespace std;

int main() {
    char ch;

    cout << "请输入一些字符: ";

    // 查看下一个字符但不读取
    ch = cin.peek();
    cout << "下一个字符是: " << ch << " " << endl;

    // 读取字符
    cin.get(ch);
    cout << "读取的字符是: " << ch << " " << endl;

    // 将字符放回输入流
    cin.putback(ch);
    cout << "字符已放回" << endl;

    // 再次读取
    cin.get(ch);
    cout << "再次读取: " << ch << " " << endl;

    return 0;
}
```

4.5.2 ignore() 方法

cpp

```
#include <iostream>
using namespace std;

int main() {
    int num;
    char ch;

    cout << "请输入一个数字和一个字符: ";
    cin >> num;
    cin.ignore();    // 忽略换行符
    cin.get(ch);

    cout << "数字: " << num << ", 字符: " << ch << "" << endl;

    return 0;
}
```

5. 文件输入和输出

5.1 文件I/O基础

5.1.1 文件流类

- `ifstream`：文件输入流
- `ofstream`：文件输出流
- `fstream`：文件输入输出流

5.1.2 基本文件写入

cpp

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    // 创建输出文件流
    ofstream outFile("data.txt");

    if (!outFile) {
        cerr << "无法创建文件! " << endl;
        return 1;
    }

    // 写入数据
    outFile << "姓名: 张三" << endl;
    outFile << "年龄: 25" << endl;
    outFile << "薪水: 5000.50" << endl;

    outFile.close();
    cout << "数据已写入文件 data.txt" << endl;

    return 0;
}
```

5.1.3 基本文件读取

cpp

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main() {
    ifstream inFile("data.txt");

    if (!inFile) {
        cerr << "无法打开文件! " << endl;
        return 1;
    }

    string line;
    cout << "文件内容: " << endl;
    while (getline(inFile, line)) {
        cout << line << endl;
    }

    inFile.close();
    return 0;
}
```

5.2 文件模式

5.2.1 常用文件模式

模式	说明
ios::in	读取模式
ios::out	写入模式
ios::app	追加模式
ios::trunc	截断模式（清空文件）
ios::binary	二进制模式
ios::ate	打开文件并定位到文件末尾

5.2.2 文件模式示例

cpp


```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    // 追加模式写入
    ofstream outFile("log.txt", ios::out | ios::app);

    if (outFile.is_open()) {
        outFile << "新的日志条目" << endl;
        outFile.close();
        cout << "日志已追加" << endl;
    }

    // 读写模式
    fstream file("data.txt", ios::in | ios::out);
    if (file.is_open()) {
        // 可以同时读写
        file << "新数据" << endl;

        file.seekg(0); // 移动到文件开头
        string line;
        while (getline(file, line)) {
            cout << line << endl;
        }

        file.close();
    }

    return 0;
}
```

5.3 二进制文件操作

5.3.1 写入二进制数据

cpp

```
#include <iostream>
#include <fstream>
using namespace std;

struct Student {
    char name[20];
    int age;
    double gpa;
};

int main() {
    Student student = {"张三", 20, 3.8};

    // 以二进制模式写入
    ofstream outFile("student.dat", ios::binary);

    if (outFile) {
        outFile.write(reinterpret_cast<char*>(&student), sizeof(Student));
        outFile.close();
        cout << "学生数据已保存到二进制文件" << endl;
    }

    return 0;
}
```

5.3.2 读取二进制数据

cpp

```
#include <iostream>
#include <fstream>
using namespace std;

struct Student {
    char name[20];
    int age;
    double gpa;
};

int main() {
    Student student;

    // 以二进制模式读取
    ifstream inFile("student.dat", ios::binary);

    if (inFile) {
        inFile.read(reinterpret_cast<char*>(&student), sizeof(Student));
        inFile.close();

        cout << "从二进制文件读取的数据: " << endl;
        cout << "姓名: " << student.name << endl;
        cout << "年龄: " << student.age << endl;
        cout << "GPA: " << student.gpa << endl;
    }

    return 0;
}
```

5.4 文件随机访问

5.4.1 文件指针操作

cpp

```

#include <iostream>
#include <fstream>
using namespace std;

int main() {
    fstream file("numbers.txt", ios::in | ios::out | ios::trunc);

    // 写入一些数据
    for (int i = 0; i < 10; i++) {
        file << i << " ";
    }

    // 移动到文件开头
    file.seekg(0, ios::beg);

    // 读取前5个数字
    int num;
    for (int i = 0; i < 5; i++) {
        file >> num;
        cout << num << " ";
    }
    cout << endl;

    // 移动到特定位置
    file.seekg(10, ios::beg); // 移动到第10个字节

    // 获取当前位置
    streampos pos = file.tellg();
    cout << "当前位置: " << pos << endl;

    file.close();
    return 0;
}

```

5.5 处理多个文件

5.5.1 合并文件

cpp

```

#include <iostream>
#include <fstream>
#include <string>
using namespace std;

void mergeFiles(const string& file1, const string& file2, const string& output) {
    ifstream in1(file1);
    ifstream in2(file2);
    ofstream out(output);

    if (!in1 || !in2 || !out) {
        cerr << "文件打开失败! " << endl;
        return;
    }

    string line;

    // 复制第一个文件的内容
    while (getline(in1, line)) {
        out << line << endl;
    }

    // 复制第二个文件的内容
    while (getline(in2, line)) {
        out << line << endl;
    }

    in1.close();
    in2.close();
    out.close();

    cout << "文件合并完成! " << endl;
}

int main() {
    mergeFiles("file1.txt", "file2.txt", "merged.txt");
    return 0;
}

```

5.6 命令行参数处理

cpp

```
#include <iostream>
#include <fstream>
using namespace std;

int main(int argc, char* argv[]) {
    if (argc != 3) {
        cout << "用法: " << argv[0] << " <输入文件> <输出文件>" << endl;
        return 1;
    }

    ifstream inFile(argv[1]);
    ofstream outFile(argv[2]);

    if (!inFile) {
        cerr << "无法打开输入文件: " << argv[1] << endl;
        return 1;
    }

    if (!outFile) {
        cerr << "无法创建输出文件: " << argv[2] << endl;
        return 1;
    }

    // 复制文件内容
    char ch;
    while (inFile.get(ch)) {
        outFile.put(ch);
    }

    inFile.close();
    outFile.close();

    cout << "文件复制完成! " << endl;
    return 0;
}
```

6. 内核格式化

6.1 字符串流 (String Streams)

字符串流允许您像操作文件一样操作字符串。

6.1.1 输出字符串流 (ostringstream)

```
#include <iostream>
#include <sstream>
#include <string>
#include <iomanip>
using namespace std;

int main() {
    ostringstream oss;

    string name = "张三";
    int age = 25;
    double salary = 5000.50;

    // 向字符串流写入格式化数据
    oss << "员工信息: " << endl;
    oss << "姓名: " << name << endl;
    oss << "年龄: " << age << endl;
    oss << "薪水: $" << fixed << setprecision(2) << salary << endl;

    // 获取格式化后的字符串
    string result = oss.str();
    cout << result << endl;

    // 也可以直接输出到文件
    ofstream file("employee.txt");
    file << result;
    file.close();

    return 0;
}
```

6.1.2 输入字符串流 (istringstream)

cpp

```
#include <iostream>
#include <sstream>
#include <string>
using namespace std;

int main() {
    string data = "张三 25 5000.50 北京";
    istringstream iss(data);

    string name, city;
    int age;
    double salary;

    // 从字符串流读取数据
    iss >> name >> age >> salary >> city;

    cout << "姓名: " << name << endl;
    cout << "年龄: " << age << endl;
    cout << "薪水: " << salary << endl;
    cout << "城市: " << city << endl;

    return 0;
}
```

6.1.3 字符串流的实际应用

1. 数字转字符串：

cpp


```

#include <iostream>
#include <sstream>
using namespace std;

string numberToString(int number) {
    ostringstream oss;
    oss << number;
    return oss.str();
}

int main() {
    int num = 12345;
    string str = numberToString(num);
    cout << "数字 " << num << " 转换为字符串: \"" << str << "\"" << endl;
    return 0;
}

```

2. 字符串转数字：

```

cpp

#include <iostream>
#include <sstream>
using namespace std;

int stringToNumber(const string& str) {
    istringstream iss(str);
    int number;
    iss >> number;
    return number;
}

int main() {
    string str = "12345";
    int num = stringToNumber(str);
    cout << "字符串 \"" << str << "\" 转换为数字: " << num << endl;
    return 0;
}

```

3. 解析CSV数据：

```

cpp

```

```
#include <iostream>
#include <sstream>
#include <string>
#include <vector>
using namespace std;

vector<string> parseCSV(const string& line) {
    vector<string> result;
    istringstream iss(line);
    string field;

    while (getline(iss, field, ',')) {
        result.push_back(field);
    }

    return result;
}

int main() {
    string csvLine = "张三,25,工程师,北京";
    vector<string> fields = parseCSV(csvLine);

    cout << "解析CSV数据: " << endl;
    for (size_t i = 0; i < fields.size(); i++) {
        cout << "字段 " << i + 1 << ": " << fields[i] << endl;
    }

    return 0;
}
```

7. 实战练习

7.1 初级练习

练习1：简单的成绩管理系统

cpp

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <iomanip>
using namespace std;

struct Student {
    string name;
    int id;
    double score;
};

class GradeManager {
private:
    vector<Student> students;
    string filename;

public:
    GradeManager(const string& file) : filename(file) {
        loadFromFile();
    }

    void addStudent() {
        Student student;
        cout << "请输入学生姓名: ";
        cin >> student.name;
        cout << "请输入学号: ";
        cin >> student.id;
        cout << "请输入成绩: ";
        cin >> student.score;

        students.push_back(student);
        cout << "学生信息已添加! " << endl;
    }

    void displayAll() {
        cout << left << setw(15) << "姓名"
            << setw(10) << "学号"
            << setw(10) << "成绩" << endl;
        cout << string(35, '-') << endl;

        for (const auto& student : students) {
            cout << left << setw(15) << student.name
                << setw(10) << student.id
                << setw(10) << fixed << setprecision(2) << student.score << endl;
        }
    }
};
```

```
}  
}  
  
void saveToFile() {  
    ofstream file(filename);  
    if (!file) {  
        cerr << "无法保存到文件！ " << endl;  
        return;  
    }  
  
    for (const auto& student : students) {  
        file << student.name << " " << student.id << " " << student.score << endl;  
    }  
    file.close();  
    cout << "数据已保存到文件！ " << endl;  
}  
  
void loadFromFile() {  
    ifstream file(filename);  
    if (!file) {  
        cout << "文件不存在，创建新的数据文件。" << endl;  
        return;  
    }  
  
    Student student;  
    while (file >> student.name >> student.id >> student.score) {  
        students.push_back(student);  
    }  
    file.close();  
    cout << "已从文件加载 " << students.size() << " 条记录。" << endl;  
}  
  
void searchStudent() {  
    string name;  
    cout << "请输入要查找的学生姓名: ";  
    cin >> name;  
  
    bool found = false;  
    for (const auto& student : students) {  
        if (student.name == name) {  
            cout << "找到学生： " << endl;  
            cout << "姓名: " << student.name << endl;  
            cout << "学号: " << student.id << endl;  
            cout << "成绩: " << student.score << endl;  
            found = true;  
            break;  
        }  
    }  
}
```

```
    }

    if (!found) {
        cout << "未找到该学生！ " << endl;
    }
}

};

int main() {
    GradeManager manager("students.txt");

    int choice;
    do {
        cout << "\n===== 成绩管理系统 =====" << endl;
        cout << "1. 添加学生" << endl;
        cout << "2. 显示所有学生" << endl;
        cout << "3. 查找学生" << endl;
        cout << "4. 保存数据" << endl;
        cout << "0. 退出" << endl;
        cout << "请选择: ";
        cin >> choice;

        switch (choice) {
            case 1:
                manager.addStudent();
                break;
            case 2:
                manager.displayAll();
                break;
            case 3:
                manager.searchStudent();
                break;
            case 4:
                manager.saveToFile();
                break;
            case 0:
                manager.saveToFile();
                cout << "程序退出！ " << endl;
                break;
            default:
                cout << "无效选择！ " << endl;
        }
    } while (choice != 0);

    return 0;
}
```

练习2：文本文件统计工具

cpp

```
#include <iostream>
#include <fstream>
#include <string>
#include <map>
#include <cctype>
using namespace std;

class TextAnalyzer {
private:
    string filename;
    int charCount;
    int wordCount;
    int lineCount;
    map<char, int> charFreq;

public:
    TextAnalyzer(const string& file) : filename(file), charCount(0), wordCount(0), lineCount(0) {}

    void analyze() {
        ifstream file(filename);
        if (!file) {
            cerr << "无法打开文件: " << filename << endl;
            return;
        }

        string line;
        bool inWord = false;

        while (getline(file, line)) {
            lineCount++;

            for (char ch : line) {
                charCount++;

                // 统计字符频率
                if (isalpha(ch)) {
                    charFreq[tolower(ch)]++;
                }

                // 统计单词数
                if (isalnum(ch)) {
                    if (!inWord) {
                        wordCount++;
                        inWord = true;
                    }
                } else {
                    inWord = false;
                }
            }
        }
    }
};
```

```

        inWord = false;
    }
}
inWord = false; // 每行结束时重置
}

file.close();
}

```

```

void displayResults() {
    cout << "\n==== 文本分析结果 =====> << endl;
    cout << "文件名: " << filename << endl;
    cout << "总字符数: " << charCount << endl;
    cout << "总单词数: " << wordCount << endl;
    cout << "总行数: " << lineCount << endl;

    cout << "\n字母频率统计:" << endl;
    for (const auto& pair : charFreq) {
        cout << pair.first << ": " << pair.second << endl;
    }
}

```

```

void saveReport(const string& reportFile) {
    ofstream report(reportFile);
    if (!report) {
        cerr << "无法创建报告文件! " << endl;
        return;
    }

    report << "文本分析报告" << endl;
    report << "=====> << endl;
    report << "源文件: " << filename << endl;
    report << "分析时间: " << __DATE__ << " " << __TIME__ << endl;
    report << endl;
    report << "统计结果:" << endl;
    report << "- 总字符数: " << charCount << endl;
    report << "- 总单词数: " << wordCount << endl;
    report << "- 总行数: " << lineCount << endl;
    report << endl;
    report << "字母频率:" << endl;

    for (const auto& pair : charFreq) {
        report << pair.first << ": " << pair.second << endl;
    }

    report.close();
    cout << "报告已保存到: " << reportFile << endl;
}

```



```

    }
};

int main(int argc, char* argv[]) {
    string inputFile, reportFile;

    if (argc > 1) {
        inputFile = argv[1];
    } else {
        cout << "请输入要分析的文件名: ";
        cin >> inputFile;
    }

    cout << "请输入报告文件名 (回车使用默认名): ";
    cin.ignore();
    getline(cin, reportFile);

    if (reportFile.empty()) {
        reportFile = "analysis_report.txt";
    }

    TextAnalyzer analyzer(inputFile);
    analyzer.analyze();
    analyzer.displayResults();
    analyzer.saveReport(reportFile);

    return 0;
}

```

7.2 中级练习

练习3：日志文件管理系统

cpp

```
#include <iostream>
#include <fstream>
#include <string>
#include <ctime>
#include <iomanip>
#include <sstream>
using namespace std;

enum LogLevel {
    INFO = 0,
    WARNING = 1,
    ERROR = 2,
    DEBUG = 3
};

class Logger {
private:
    string logFile;
    LogLevel minLevel;

    string getCurrentTime() {
        time_t now = time(0);
        tm* timeInfo = localtime(&now);

        ostringstream oss;
        oss << put_time(timeInfo, "%Y-%m-%d %H:%M:%S");
        return oss.str();
    }

    string levelToString(LogLevel level) {
        switch (level) {
            case INFO: return "INFO";
            case WARNING: return "WARNING";
            case ERROR: return "ERROR";
            case DEBUG: return "DEBUG";
            default: return "UNKNOWN";
        }
    }

public:
    Logger(const string& filename, LogLevel level = INFO)
        : logFile(filename), minLevel(level) {}

    void log(LogLevel level, const string& message) {
        if (level < minLevel) return;
```

```
ofstream file(logFile, ios::app);
if (!file) {
    cerr << "无法写入日志文件! " << endl;
    return;
}

file << "[" << getCurrentTime() << " ] "
    << "[" << levelToString(level) << " ] "
    << message << endl;

file.close();
}

void info(const string& message) {
    log(INFO, message);
}

void warning(const string& message) {
    log(WARNING, message);
}

void error(const string& message) {
    log(ERROR, message);
}

void debug(const string& message) {
    log(DEBUG, message);
}

void displayLogs() {
    ifstream file(logFile);
    if (!file) {
        cout << "日志文件不存在! " << endl;
        return;
    }

    string line;
    cout << "\n===== 日志记录 =====" << endl;
    while (getline(file, line)) {
        cout << line << endl;
    }
    file.close();
}

void clearLogs() {
    ofstream file(logFile, ios::trunc);
    file.close();
}
```

```

        cout << "日志已清空! " << endl;
    }

    void filterLogs(LogLevel level) {
        ifstream file(logFile);
        if (!file) {
            cout << "日志文件不存在! " << endl;
            return;
        }

        string line;
        string levelStr = "[" + levelToString(level) + "];";

        cout << "\n==== " << levelToString(level) << " 级别日志 =====" << endl;
        while (getline(file, line)) {
            if (line.find(levelStr) != string::npos) {
                cout << line << endl;
            }
        }
        file.close();
    }
};

```

// 使用示例

```

int main() {
    Logger logger("system.log", DEBUG);

    int choice;
    do {
        cout << "\n==== 日志管理系统 =====" << endl;
        cout << "1. 记录INFO日志" << endl;
        cout << "2. 记录WARNING日志" << endl;
        cout << "3. 记录ERROR日志" << endl;
        cout << "4. 记录DEBUG日志" << endl;
        cout << "5. 显示所有日志" << endl;
        cout << "6. 按级别筛选日志" << endl;
        cout << "7. 清空日志" << endl;
        cout << "0. 退出" << endl;
        cout << "请选择: ";
        cin >> choice;
        cin.ignore(); // 清除换行符

        string message;
        switch (choice) {
            case 1:
                cout << "请输入INFO消息: ";
                getline(cin, message);

```

```

        logger.info(message);
        break;
    case 2:
        cout << "请输入WARNING消息: ";
        getline(cin, message);
        logger.warning(message);
        break;
    case 3:
        cout << "请输入ERROR消息: ";
        getline(cin, message);
        logger.error(message);
        break;
    case 4:
        cout << "请输入DEBUG消息: ";
        getline(cin, message);
        logger.debug(message);
        break;
    case 5:
        logger.displayLogs();
        break;
    case 6: {
        int levelChoice;
        cout << "选择级别 (0=INFO, 1=WARNING, 2=ERROR, 3=DEBUG): ";
        cin >> levelChoice;
        logger.filterLogs(static_cast<LogLevel>(levelChoice));
        break;
    }
    case 7:
        logger.clearLogs();
        break;
    case 0:
        cout << "程序退出! " << endl;
        break;
    default:
        cout << "无效选择! " << endl;
    }
} while (choice != 0);

return 0;
}

```

7.3 高级练习

练习4：简单数据库系统

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <map>
#include <sstream>
#include <algorithm>
using namespace std;

class SimpleDatabase {
private:
    string dbFile;
    vector<map<string, string>> records;
    vector<string> fields;

public:
    SimpleDatabase(const string& filename) : dbFile(filename) {
        loadDatabase();
    }

    ~SimpleDatabase() {
        saveDatabase();
    }

    void createTable(const vector<string>& fieldNames) {
        fields = fieldNames;
        records.clear();
        cout << "表已创建，字段: ";
        for (const auto& field : fields) {
            cout << field << " ";
        }
        cout << endl;
    }

    void insertRecord() {
        if (fields.empty()) {
            cout << "请先创建表结构! " << endl;
            return;
        }

        map<string, string> record;

        for (const auto& field : fields) {
            string value;
            cout << "请输入 " << field << ": ";
            getline(cin, value);
```

```
    record[field] = value;
}

records.push_back(record);
cout << "记录已添加! " << endl;
}

void displayAllRecords() {
    if (records.empty()) {
        cout << "没有记录! " << endl;
        return;
    }

    // 显示表头
    for (const auto& field : fields) {
        cout << left << setw(15) << field;
    }
    cout << endl;
    cout << string(fields.size() * 15, '-') << endl;

    // 显示记录
    for (const auto& record : records) {
        for (const auto& field : fields) {
            cout << left << setw(15) << record.at(field);
        }
        cout << endl;
    }
}

void searchRecords(const string& field, const string& value) {
    bool found = false;

    cout << "搜索结果:" << endl;
    for (const auto& field_name : fields) {
        cout << left << setw(15) << field_name;
    }
    cout << endl;
    cout << string(fields.size() * 15, '-') << endl;

    for (const auto& record : records) {
        if (record.count(field) && record.at(field) == value) {
            for (const auto& field_name : fields) {
                cout << left << setw(15) << record.at(field_name);
            }
            cout << endl;
            found = true;
        }
    }
}
```

```

}

if (!found) {
    cout << "未找到匹配的记录! " << endl;
}
}

void deleteRecord(const string& field, const string& value) {
    auto it = remove_if(records.begin(), records.end(),
        [&field, &value](const map<string, string>& record) {
            return record.count(field) && record.at(field) == value;
        });

    int deletedCount = records.end() - it;
    records.erase(it, records.end());

    cout << "已删除 " << deletedCount << " 条记录! " << endl;
}

void saveDatabase() {
    ofstream file(dbFile);
    if (!file) {
        cerr << "无法保存数据库! " << endl;
        return;
    }

    // 保存字段名
    for (size_t i = 0; i < fields.size(); i++) {
        file << fields[i];
        if (i < fields.size() - 1) file << ",";
    }
    file << endl;

    // 保存记录
    for (const auto& record : records) {
        for (size_t i = 0; i < fields.size(); i++) {
            file << record.at(fields[i]);
            if (i < fields.size() - 1) file << ",";
        }
        file << endl;
    }

    file.close();
}

void loadDatabase() {
    ifstream file(dbFile);

```



```

if (!file) {
    cout << "数据库文件不存在，将创建新文件。" << endl;
    return;
}

string line;
if (getline(file, line)) {
    // 解析字段名
    stringstream ss(line);
    string field;
    while (getline(ss, field, ',')) {
        fields.push_back(field);
    }
}

// 解析记录
while (getline(file, line)) {
    stringstream ss(line);
    string value;
    map<string, string> record;

    for (size_t i = 0; i < fields.size() && getline(ss, value, ','); i++) {
        record[fields[i]] = value;
    }

    if (!record.empty()) {
        records.push_back(record);
    }
}

file.close();
cout << "已加载 " << records.size() << " 条记录。" << endl;
}

};

int main() {
    SimpleDatabase db("database.csv");

    int choice;
    do {
        cout << "\n===== 简单数据库系统 =====" << endl;
        cout << "1. 创建表" << endl;
        cout << "2. 插入记录" << endl;
        cout << "3. 显示所有记录" << endl;
        cout << "4. 搜索记录" << endl;
        cout << "5. 删除记录" << endl;
        cout << "6. 保存数据库" << endl;
    } while (choice < 7);
}

```

```
cout << "0. 退出" << endl;
cout << "请选择: ";
cin >> choice;
cin.ignore();

switch (choice) {
    case 1: {
        cout << "请输入字段数量: ";
        int fieldCount;
        cin >> fieldCount;
        cin.ignore();

        vector<string> fieldNames;
        for (int i = 0; i < fieldCount; i++) {
            string fieldName;
            cout << "请输入字段 " << (i+1) << " 的名称: ";
            getline(cin, fieldName);
            fieldNames.push_back(fieldName);
        }

        db.createTable(fieldNames);
        break;
    }
    case 2:
        db.insertRecord();
        break;
    case 3:
        db.displayAllRecords();
        break;
    case 4: {
        string field, value;
        cout << "请输入搜索字段: ";
        getline(cin, field);
        cout << "请输入搜索值: ";
        getline(cin, value);
        db.searchRecords(field, value);
        break;
    }
    case 5: {
        string field, value;
        cout << "请输入删除条件字段: ";
        getline(cin, field);
        cout << "请输入删除条件值: ";
        getline(cin, value);
        db.deleteRecord(field, value);
        break;
    }
}
```

```

    case 6:
        db.saveDatabase();
        cout << "数据库已保存！" << endl;
        break;
    case 0:
        cout << "程序退出！" << endl;
        break;
    default:
        cout << "无效选择！" << endl;
}
} while (choice != 0);

return 0;
}

```

8. 常见问题解答

8.1 输入输出常见问题

Q1: 为什么使用 读取字符串时只能读到第一个单词？ `cin >>`

A1: 遇到空白字符（空格、制表符、换行符）就会停止读取。如果要读取包含空格的字符串，应该使用：`cin >>` `getline()`

```

cpp

string fullName;
cout << "请输入全名: ";
getline(cin, fullName); // 可以读取包含空格的字符串

```

Q2: 为什么在使用 后再使用 会出现问题？ `cin >>` `getline()`

A2: 不会读取换行符，它会留在输入缓冲区中。解决方法是在使用 前先清除换行符：`cin >>` `getline()`

```

CPP

int age;
string name;

cin >> age;
cin.ignore(); // 清除换行符
getline(cin, name);

```

Q3: 如何检查文件是否成功打开？

A3: 使用方法或者直接检查流对象：`is_open()`

CPP

```
ifstream file("data.txt");

// 方法1
if (!file.is_open()) {
    cerr << "文件打开失败! " << endl;
    return 1;
}

// 方法2
if (!file) {
    cerr << "文件打开失败! " << endl;
    return 1;
}
```

Q4: 二进制文件和文本文件有什么区别?

答4:

- **文本文件**: 以字符形式存储数据, 人类可读, 跨平台兼容性好
- **二进制文件**: 以二进制形式存储数据, 节省空间, 读写速度快, 但不可直接阅读

CPP

```
// 文本文件
ofstream textFile("data.txt");
textFile << 42 << " " << 3.14; // 存储为字符 "42 3.14"

// 二进制文件
ofstream binFile("data.bin", ios::binary);
int num = 42;
double pi = 3.14;
binFile.write(reinterpret_cast<char*>(&num), sizeof(int));
binFile.write(reinterpret_cast<char*>(&pi), sizeof(double));
```

8.2 格式化输出问题

Q5: 如何让数字右对齐并用0填充?

A5: 使用 和 : `setfill()` `setw()`

CPP

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    int num = 42;
    cout << setfill('0') << setw(6) << num << endl; // 输出: 000042
    return 0;
}
```

Q6: 如何控制浮点数的显示精度?

A6: 使用和格式标志: `setprecision()`

```
CPP

double pi = 3.14159265;

cout << pi << endl; // 默认精度
cout << setprecision(3) << pi << endl; // 3位精度
cout << fixed << setprecision(2) << pi << endl; // 固定2位小数
cout << scientific << pi << endl; // 科学计数法
```

8.3 文件操作问题

Q7: 如何在文件末尾追加内容而不覆盖原内容?

A7: 使用模式: `ios::app`

```
CPP

ofstream file("log.txt", ios::app);
file << "新的日志条目" << endl;
```

Q8: 如何读取文件的所有内容到字符串中?

A8: 几种方法:

```
CPP
```

```

// 方法1: 逐行读取
ifstream file("data.txt");
string content, line;
while (getline(file, line)) {
    content += line + "\n";
}

// 方法2: 使用字符串流
ifstream file("data.txt");
stringstream buffer;
buffer << file.rdbuf();
string content = buffer.str();

// 方法3: 使用迭代器
ifstream file("data.txt");
string content((istreambuf_iterator<char>(file),
    istreambuf_iterator<char>()));

```

8.4 性能优化建议

Q9: 如何提高文件I/O的性能?

A9: 几个优化建议:

1. 使用适当的缓冲区大小
2. 减少频繁的文件打开/关闭操作
3. 批量读写而不是逐字符操作
4. 对于大文件，考虑使用二进制模式

```

CPP

// 好的做法: 批量读取
ifstream file("large_file.txt");
const int BUFFER_SIZE = 8192;
char buffer[BUFFER_SIZE];

while (file.read(buffer, BUFFER_SIZE) || file.gcount() > 0) {
    // 处理缓冲区中的数据
    int bytesRead = file.gcount();
    // ...
}

```

Q10: 什么时候应该使用字符串流?

A10: 字符串流适用于:

1. 格式化字符串构建
2. 数据类型转换
3. 解析复杂的字符串数据
4. 临时缓冲和数据处理

C++

```
// 构建复杂的格式化字符串
ostringstream oss;
oss << "用户 " << username << " 在 " << timestamp
    << " 执行了操作, 结果代码: " << resultCode;
string logMessage = oss.str();
```

总结

这份C++输入输出与文件处理自学手册涵盖了：

核心概念

- 流和缓冲区的工作原理
- C++输入输出类库的层次结构
- 标准流对象的使用方法

基本操作

- `cout` 输出格式化技巧
- `cin` 输入处理和验证
- 错误处理和流状态管理

文件操作

- 文本文件和二进制文件的读写
- 文件模式和随机访问
- 高级文件操作技巧

实战应用

- 完整的项目实例
- 常见问题的解决方案
- 性能优化建议

学习建议

1. **从基础开始**：先掌握基本的输入输出
2. **多写代码**：通过实际编程来巩固概念
3. **循序渐进**：从简单的例子开始，逐步处理复杂的问题
4. **错误处理**：养成良好的错误检查习惯
5. **实践项目**：尝试完成手册中的练习项目

掌握C++的输入输出和文件处理是成为优秀C++程序员的必经之路。通过这份手册的学习和实践，您将能够熟练地处理各种数据输入输出需求，为进一步学习C++的高级特性打下坚实的基础。

记住：编程是一门实践性很强的技能，多动手练习比单纯阅读更加重要！